ES Lab Exam: Complete Guide

Arduino + Proteus. Every program: parts, full connections, test and code on one page.

Contents

0	Smart Door Lock (your assignment)
1	Temperature monitor: LM35 + LCD + alert
2	Automatic street light: LDR
3a / 3b	7-segment counter / with push button
4	Traffic signal
5a / 5b / 5c	Blink with millis / running lights / PWM with pot
6 / 6b	Keypad calculator / same with switch-case
7	Counter on LCD with push button
After that	The big picture: which Arduino, every part, wiring rules, change tasks, viva Q&A

Making any program in Proteus (built-in Arduino, code in Source Code tab)

1. Open Proteus 8 Professional. On the Home page, under Start, click New Project.
2. Start page: type a Name, Browse to your folder, keep New Project selected, click Next.
3. Schematic Design: Create a schematic from the selected template, DEFAULT, Next.
4. PCB Layout: Do not create a PCB layout, Next.
5. Firmware: Create Firmware Project. Family ARDUINO, Controller Arduino Uno, Compiler Arduino AVR (Proteus). Next, then Finish.
6. Schematic Capture tab: press P, type each part name, double-click it, OK. Click a part in DEVICES, then click the sheet to place it.
7. Terminals Mode (left toolbar): place POWER (+5V) and GROUND terminals where the connection table says +5V or GND.
8. Wire: click the end of one pin (green pencil), then the end of the other. Double-click a resistor to set 220.
9. Source Code tab: click in main.ino, Ctrl+A, paste the code. Then Build > Build Project; the output must show no errors.
10. Back to Schematic Capture, click Play (bottom left). Stop to stop. Ctrl+S saves circuit and code in one .pdsprj file.

Pin names

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). The code uses plain numbers (8, A0). Blue "ARDUINO UNO V3 by TEP" board: labels are the plain numbers. It has no Source Code tab: compile in the Arduino IDE (Sketch > Export Compiled Binary), then double-click the board and set Program File to the .hex.

The one rule

Every pin number in the code is a wire to that pin. If you move a wire, change the number in the code too. A part needs both ends connected (signal and GND) to work.

0. Smart Door Lock

Parts: ARDUINO UNO, LM016L, KEYPAD-PHONE, MOTOR-PWMSERVO, BUZZER, LED-RED, LED-GREEN, RES x2

Connections

LCD VSS	**GND**	LCD D7	**A0**	Servo + (top)	**+5V**
LCD VDD	**+5V (POWER)**	LCD D0-D3	**not connected**	Servo signal (mid)	**9**
LCD VEE	**GND**	Keypad row A	**0**	Servo - (bottom)	**GND**
LCD RS	**A4**	Keypad row B	**1**	Buzzer +	**8**
LCD RW	**GND**	Keypad row C	**2**	Buzzer -	**GND**
LCD E	**A5**	Keypad row D	**3**	Red LED anode	**pin 10 via RES 220**
LCD D4	**A3**	Keypad col 1	**4**	Red LED cathode	**GND**
LCD D5	**A2**	Keypad col 2	**5**	Green LED anode	**pin 11 via RES 220**
LCD D6	**A1**	Keypad col 3	**6**	Green LED cathode	**GND**

*Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: 1 2 3 4 # gives ACCESS GRANTED and the servo turns. * clears. 3 wrong tries block entry 10 s.*

```
#include <LiquidCrystal.h>
#include <Servo.h>
LiquidCrystal lcd(A4, A5, A3, A2, A1, A0);   // RS, E, D4, D5, D6, D7
Servo door;
char keys[4][3] = {{'1','2','3'}, {'4','5','6'}, {'7','8','9'}, {'*','0','#'}};
int rowPin[4] = {0, 1, 2, 3};   // rows A B C D
int colPin[3] = {4, 5, 6};      // columns 1 2 3
int buzzer = 8, redLed = 10, greenLed = 11;
char pass[] = "1234";
char entered[5];
int len = 0, attempts = 0;
char getKey() {
  for (int r = 0; r < 4; r++) {
    digitalWrite(rowPin[r], LOW);
    for (int c = 0; c < 3; c++) {
      if (digitalRead(colPin[c]) == LOW) {
        delay(30);
        while (digitalRead(colPin[c]) == LOW);   // wait for release
        digitalWrite(rowPin[r], HIGH);
        return keys[r][c];
      }
    }
    digitalWrite(rowPin[r], HIGH);
  }
  return 0;
}
void prompt() { len = 0; lcd.clear(); lcd.print("Password: "); }
void setup() {
  lcd.begin(16, 2);
  door.attach(9);
  door.write(0);                                  // locked
  pinMode(buzzer, OUTPUT); pinMode(redLed, OUTPUT); pinMode(greenLed, OUTPUT);
  digitalWrite(redLed, HIGH);
  for (int i = 0; i < 4; i++) { pinMode(rowPin[i], OUTPUT); digitalWrite(rowPin[i], HIGH); }
  for (int i = 0; i < 3; i++) pinMode(colPin[i], INPUT_PULLUP);
  lcd.print("SMART DOOR LOCK");
  delay(1500);
  prompt();
}
void loop() {
  char k = getKey();
  if (k == 0) return;
  if (k == '*') { prompt(); return; }             // clear
  if (k >= '0' && k <= '9' && len < 4) { entered[len++] = k; lcd.print('*'); return; }
  if (k != '#') return;
  entered[len] = '\0';
  lcd.clear();
  if (strcmp(entered, pass) == 0) {               // correct
    attempts = 0;
    lcd.print("ACCESS GRANTED");
    digitalWrite(redLed, LOW); digitalWrite(greenLed, HIGH);
    door.write(90);                               // unlock
    delay(5000);
    door.write(0);                                // auto lock
    digitalWrite(greenLed, LOW); digitalWrite(redLed, HIGH);
  } else {                                        // wrong
    attempts++;
    lcd.print("ACCESS DENIED");
    digitalWrite(buzzer, HIGH); delay(1000); digitalWrite(buzzer, LOW);
    if (attempts >= 3) {
      lcd.clear(); lcd.print("SYSTEM LOCKED");
      lcd.setCursor(0, 1); lcd.print("Wait 10 seconds");
      delay(10000);                               // entry disabled
      attempts = 0;
    }
  }
  prompt();
}
```

1. Temperature monitor (LM35 + LCD + alert)

Parts: ARDUINO UNO, LM016L, LM35, BUZZER, LED-RED, RES

Connections

LCD VSS	**GND**	LCD D7	**7**
LCD VDD	**+5V (POWER)**	LCD D0-D3	**not connected**
LCD VEE	**GND**	LM35 +VS	**+5V**
LCD RS	**A4**	LM35 VOUT	**A0**
LCD RW	**GND**	LM35 GND	**GND**
LCD E	**A5**	Buzzer +	**8**
LCD D4	**A3**	Buzzer -	**GND**
LCD D5	**A2**	Red LED anode	**pin 10 via RES 220**
LCD D6	**A1**	Red LED cathode	**GND**

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: click the LM35 arrows. Above 30 C the LED and buzzer turn on and the LCD shows ALERT.

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(A4, A5, A3, A2, A1, 7);   // RS, E, D4, D5, D6, D7
int led = 10;
int buzzer = 8;
float limit = 30.0;
void setup() {
  lcd.begin(16, 2);
  pinMode(led, OUTPUT);
  pinMode(buzzer, OUTPUT);
}
void loop() {
  int raw = analogRead(A0);
  float t = raw * 500.0 / 1023;   // LM35: 10 mV per degree C
  lcd.setCursor(0, 0);
  lcd.print("Temp: ");
  lcd.print(t, 1);
  lcd.print((char)223);           // degree symbol
  lcd.print("C   ");
  lcd.setCursor(0, 1);
  if (t > limit) {
    lcd.print("ALERT: HIGH     ");
    digitalWrite(led, HIGH);
    digitalWrite(buzzer, HIGH);
  } else {
    lcd.print("Status: NORMAL  ");
    digitalWrite(led, LOW);
    digitalWrite(buzzer, LOW);
  }
  delay(500);
}
```

2. Automatic street light (LDR)

Parts: ARDUINO UNO, LM016L, TORCH_LDR, RES 10k, LED-RED, RES 220

Connections

LCD VSS	**GND**	LCD D7	**7**
LCD VDD	**+5V (POWER)**	LCD D0-D3	**not connected**
LCD VEE	**GND**	RES 10k end 1	**+5V**
LCD RS	**A4**	RES 10k end 2	**A0**
LCD RW	**GND**	LDR end 1	**A0**
LCD E	**A5**	LDR end 2	**GND**
LCD D4	**A3**	Red LED anode	**pin 10 via RES 220**
LCD D5	**A2**	Red LED cathode	**GND**
LCD D6	**A1**		

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: darken the TORCH_LDR with its arrows. Above 500 the LED turns on. Backwards? Swap the LDR and the 10k.

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(A4, A5, A3, A2, A1, 7);
int led = 10;
int limit = 500;          // above this = dark
void setup() {
  lcd.begin(16, 2);
  pinMode(led, OUTPUT);
}
void loop() {
  int v = analogRead(A0);
  lcd.setCursor(0, 0);
  lcd.print("Light val: ");
  lcd.print(v);
  lcd.print("    ");
  lcd.setCursor(0, 1);
  if (v > limit) {
    digitalWrite(led, HIGH);
    lcd.print("Night: LIGHT ON ");
  } else {
    digitalWrite(led, LOW);
    lcd.print("Day: LIGHT OFF  ");
  }
  delay(300);
}
```

3a. 7-segment counter 0 to 9 and back

Parts: ARDUINO UNO, 7SEG-COM-CATHODE

Connections

Segment A	**0**	Segment F	**5**
Segment B	**1**	Segment G	**6**
Segment C	**2**	Segment DP	**not connected**
Segment D	**3**	Common pin	**GND**
Segment E	**4**		

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: shows 0, 1 ... 9, then 9 ... 0, one digit per second.

```
byte digit[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
int segPin[7] = {0, 1, 2, 3, 4, 5, 6};   // a b c d e f g
void show(int n) {
  for (int i = 0; i < 7; i++) {
    digitalWrite(segPin[i], bitRead(digit[n], i));   // bit i drives segment i
  }
}
void setup() {
  for (int i = 0; i < 7; i++) pinMode(segPin[i], OUTPUT);
}
void loop() {
  for (int n = 0; n <= 9; n++) { show(n); delay(1000); }   // 0 to 9
  for (int n = 9; n >= 0; n--) { show(n); delay(1000); }   // 9 to 0
}
```

3b. 7-segment counter with push button

Parts: ARDUINO UNO, 7SEG-COM-CATHODE, BUTTON

Connections

Segment A	**0**	Segment F	**5**
Segment B	**1**	Segment G	**6**
Segment C	**2**	Common pin	**GND**
Segment D	**3**	Button side 1	**7**
Segment E	**4**	Button side 2	**GND**

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: each click adds 1; after 9 it goes back to 0.

```
byte digit[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
int segPin[7] = {0, 1, 2, 3, 4, 5, 6};   // a b c d e f g
void show(int n) {
  for (int i = 0; i < 7; i++) {
    digitalWrite(segPin[i], bitRead(digit[n], i));   // bit i drives segment i
  }
}
int count = 0;
void setup() {
  for (int i = 0; i < 7; i++) pinMode(segPin[i], OUTPUT);
  pinMode(7, INPUT_PULLUP);
  show(0);
}
void loop() {
  if (digitalRead(7) == LOW) {          // pressed
    count = (count + 1) % 10;           // 9 wraps to 0
    show(count);
    delay(50);
    while (digitalRead(7) == LOW);      // wait for release
    delay(50);
  }
}
```

4. Traffic signal

Parts: ARDUINO UNO, LED-RED, LED-YELLOW, LED-GREEN, RES x3

Connections

Red LED anode	**pin 10 via RES 220**
Red LED cathode	**GND**
Yellow LED anode	**pin 12 via RES 220**
Yellow LED cathode	**GND**
Green LED anode	**pin 11 via RES 220**
Green LED cathode	**GND**

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: red 5 s, green 5 s, yellow 2 s, repeat.

```
int red = 10, yellow = 12, green = 11;
void setup() {
  pinMode(red, OUTPUT);
  pinMode(yellow, OUTPUT);
  pinMode(green, OUTPUT);
}
void light(int r, int y, int g, int ms) {
  digitalWrite(red, r);
  digitalWrite(yellow, y);
  digitalWrite(green, g);
  delay(ms);
}
void loop() {
  light(1, 0, 0, 5000);   // red 5 s
  light(0, 0, 1, 5000);   // green 5 s
  light(0, 1, 0, 2000);   // yellow 2 s
}
```

5a. LED blink without delay() (millis)

Parts: ARDUINO UNO, LED-RED, RES

Connections

Red LED anode	**pin 10 via RES 220**
Red LED cathode	**GND**

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: the LED blinks every half second.

```
int led = 10;
unsigned long last = 0;
int state = LOW;
void setup() { pinMode(led, OUTPUT); }
void loop() {
  if (millis() - last >= 500) {   // 500 ms passed?
    last = millis();
    state = !state;
    digitalWrite(led, state);
  }
}
```

5b. Running lights (8 LEDs)

Parts: ARDUINO UNO, LED-RED x8, RES x8

Connections

LED 1 anode	**pin 0 via RES 220**	LED 6 anode	**pin 5 via RES 220**
LED 2 anode	**pin 1 via RES 220**	LED 7 anode	**pin 6 via RES 220**
LED 3 anode	**pin 2 via RES 220**	LED 8 anode	**pin 7 via RES 220**
LED 4 anode	**pin 3 via RES 220**	All 8 LED cathodes	**GND**
LED 5 anode	**pin 4 via RES 220**		

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: one light runs from pin 0 to pin 7 and back.

```
void setup() {
  for (int i = 0; i <= 7; i++) pinMode(i, OUTPUT);
}
void loop() {
  for (int i = 0; i <= 7; i++) { digitalWrite(i, HIGH); delay(150); digitalWrite(i, LOW); }  // left to right
  for (int i = 6; i >= 1; i--) { digitalWrite(i, HIGH); delay(150); digitalWrite(i, LOW); }  // back
}
```

5c. LED brightness with potentiometer (PWM)

Parts: ARDUINO UNO, POT-HG, LED-RED, RES

Connections

POT end 1	**+5V**
POT end 2	**GND**
POT middle (wiper)	**A0**
Red LED anode	**pin 10 via RES 220**
Red LED cathode	**GND**

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: click the POT-HG arrows; the LED gets brighter and dimmer.

```
int led = 10;                       // must be a PWM pin: 3, 5, 6, 9, 10, 11
void setup() { pinMode(led, OUTPUT); }
void loop() {
  int v = analogRead(A0);           // 0 to 1023
  int b = map(v, 0, 1023, 0, 255);  // analogWrite takes 0 to 255
  analogWrite(led, b);
}
```

6. Keypad calculator with LCD

Parts: ARDUINO UNO, LM016L, KEYPAD-SMALLCALC

Connections

LCD VSS	**GND**	LCD D0-D3	**not connected**
LCD VDD	**+5V (POWER)**	Keypad row A	**0**
LCD VEE	**GND**	Keypad row B	**1**
LCD RS	**A4**	Keypad row C	**2**
LCD RW	**GND**	Keypad row D	**3**
LCD E	**A5**	Keypad col 1	**4**
LCD D4	**A3**	Keypad col 2	**5**
LCD D5	**A2**	Keypad col 3	**6**
LCD D6	**A1**	Keypad col 4	**7**
LCD D7	**A0**		

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: press 1 2 + 3 4 = ; the LCD shows 12+34, then = 46. ON/C clears.

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(A4, A5, A3, A2, A1, A0);   // RS, E, D4, D5, D6, D7
char keys[4][4] = {
  {'7', '8', '9', '/'},
  {'4', '5', '6', '*'},
  {'1', '2', '3', '-'},
  {'C', '0', '=', '+'}
};
int rowPin[4] = {0, 1, 2, 3};
int colPin[4] = {4, 5, 6, 7};
long a = 0, b = 0;
char op = 0;
int done = 0;
char getKey() {
  for (int r = 0; r < 4; r++) {
    digitalWrite(rowPin[r], LOW);              // activate one row
    for (int c = 0; c < 4; c++) {
      if (digitalRead(colPin[c]) == LOW) {     // key at (r, c) pressed
        delay(30);
        while (digitalRead(colPin[c]) == LOW); // wait for release
        digitalWrite(rowPin[r], HIGH);
        return keys[r][c];
      }
    }
    digitalWrite(rowPin[r], HIGH);
  }
  return 0;                                    // no key
}
void clearAll() {
  a = 0; b = 0; op = 0; done = 0;
  lcd.clear();
}
void setup() {
  lcd.begin(16, 2);
  for (int i = 0; i < 4; i++) {
    pinMode(rowPin[i], OUTPUT);
    digitalWrite(rowPin[i], HIGH);
    pinMode(colPin[i], INPUT_PULLUP);
  }
  clearAll();
}
void loop() {
  char k = getKey();
  if (k == 0) return;
  if (k == 'C') { clearAll(); return; }
  if (done) clearAll();                        // new sum after an answer
  if (k >= '0' && k <= '9') {
    if (op == 0) a = a * 10 + (k - '0');       // building first number
    else         b = b * 10 + (k - '0');       // building second number
    lcd.print(k);
  }
  else if (k == '=') {
    lcd.setCursor(0, 1);
    if (op == '/' && b == 0) lcd.print("Error");
    else {
      long r = a;
      if (op == '+') r = a + b;
      if (op == '-') r = a - b;
      if (op == '*') r = a * b;
      if (op == '/') r = a / b;
      lcd.print("= ");
      lcd.print(r);
    }
    done = 1;
  }
  else if (op == 0) {                          // + - * /
    op = k;
    lcd.print(k);
  }
}
```

6b. Calculator using switch-case

Parts: ARDUINO UNO, LM016L, KEYPAD-SMALLCALC (same circuit as page 6)

Connections

LCD VSS	**GND**	LCD D0-D3	**not connected**
LCD VDD	**+5V (POWER)**	Keypad row A	**0**
LCD VEE	**GND**	Keypad row B	**1**
LCD RS	**A4**	Keypad row C	**2**
LCD RW	**GND**	Keypad row D	**3**
LCD E	**A5**	Keypad col 1	**4**
LCD D4	**A3**	Keypad col 2	**5**
LCD D5	**A2**	Keypad col 3	**6**
LCD D6	**A1**	Keypad col 4	**7**
LCD D7	**A0**		

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Same as page 6, but the arithmetic is done with switch-case in calc(). Use this if they ask for switch-case.

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(A4, A5, A3, A2, A1, A0);   // RS, E, D4, D5, D6, D7
char keys[4][4] = {
  {'7', '8', '9', '/'},
  {'4', '5', '6', '*'},
  {'1', '2', '3', '-'},
  {'C', '0', '=', '+'}
};
int rowPin[4] = {0, 1, 2, 3};
int colPin[4] = {4, 5, 6, 7};
long a = 0, b = 0;
char op = 0;
int done = 0;
char getKey() {
  for (int r = 0; r < 4; r++) {
    digitalWrite(rowPin[r], LOW);              // activate one row
    for (int c = 0; c < 4; c++) {
      if (digitalRead(colPin[c]) == LOW) {     // key at (r, c) pressed
        delay(30);
        while (digitalRead(colPin[c]) == LOW); // wait for release
        digitalWrite(rowPin[r], HIGH);
        return keys[r][c];
      }
    }
    digitalWrite(rowPin[r], HIGH);
  }
  return 0;                                    // no key
}
long calc(long a, long b, char op) {
  long r = a;
  switch (op) {
    case '+': r = a + b; break;
    case '-': r = a - b; break;
    case '*': r = a * b; break;
    case '/': r = a / b; break;
  }
  return r;
}
void clearAll() {
  a = 0; b = 0; op = 0; done = 0;
  lcd.clear();
}
void setup() {
  lcd.begin(16, 2);
  for (int i = 0; i < 4; i++) {
    pinMode(rowPin[i], OUTPUT);
    digitalWrite(rowPin[i], HIGH);
    pinMode(colPin[i], INPUT_PULLUP);
  }
  clearAll();
}
void loop() {
  char k = getKey();
  if (k == 0) return;
  if (k == 'C') { clearAll(); return; }
  if (done) clearAll();                        // new sum after an answer
  if (k >= '0' && k <= '9') {
    if (op == 0) a = a * 10 + (k - '0');       // building first number
    else         b = b * 10 + (k - '0');       // building second number
    lcd.print(k);
  }
  else if (k == '=') {
    lcd.setCursor(0, 1);
    if (op == '/' && b == 0) lcd.print("Error");
    else {
      long r = calc(a, b, op);              // switch-case below
      lcd.print("= ");
      lcd.print(r);
    }
    done = 1;
  }
  else if (op == 0) {                          // + - * /
    op = k;
    lcd.print(k);
  }
}
```

7. Counter on LCD with push button

Parts: ARDUINO UNO, LM016L, BUTTON

Connections

LCD VSS	**GND**	LCD D5	**A2**
LCD VDD	**+5V (POWER)**	LCD D6	**A1**
LCD VEE	**GND**	LCD D7	**A0**
LCD RS	**A4**	LCD D0-D3	**not connected**
LCD RW	**GND**	Button side 1	**7**
LCD E	**A5**	Button side 2	**GND**
LCD D4	**A3**		

Built-in Proteus Arduino: pin 8 = IO8, A0 = AD0 (same for every number). Blue TEP board: labels match these numbers. Test: each button click adds 1 and the LCD shows the new count on line 2.

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(A4, A5, A3, A2, A1, A0);   // RS, E, D4, D5, D6, D7
int button = 7;
int count = 0;
void setup() {
  lcd.begin(16, 2);
  pinMode(button, INPUT_PULLUP);
  lcd.print("Count:");
}
void loop() {
  if (digitalRead(button) == LOW) {          // pressed
    count++;
    lcd.setCursor(0, 1);
    lcd.print(count);
    lcd.print("    ");
    delay(50);
    while (digitalRead(button) == LOW);      // wait for release
    delay(50);
  }
}
```

ES Lab: The Big Picture (Circuit, Connections, Viva)

Nov 11, 2026 · @Sai

Blue board or built-in Arduino: does it matter?

No. Both are the same chip, the ATmega328P, and the same code runs on both. Only two things differ: how the board looks, and how you load the code.

Which one you used: your Assignment 10 circuit uses the **built-in Proteus ARDUINO UNO**. It's the plain box with the ATmega328P inside a dashed red border, labelled "Arduino 328", with pins named IO0 to IO13 and AD0 to AD5. The circuit you sent last night used the **blue "ARDUINO UNO V3 by TEP"** board. Either is fine in the exam.

	Built-in ARDUINO UNO (your assignment)	Blue ARDUINO UNO V3 (TEP)
Looks like	Box with the ATmega328P, dashed border	Photo-like blue board
Digital pin names	IO0 to IO13	0 to 13
Analog pin names	AD0 to AD5	A0 to A5
How you get it	File → New Project → Firmware page → Arduino Uno	Press P, search ARDUINO UNO (only if the lab installed the TEP library)
Where the code goes	Proteus **Source Code** tab, then Build → Build Project	Arduino IDE → Sketch → Export Compiled Binary → double-click board → **Program File** = the .hex
Power wiring for the board	None needed	None needed

The one rule that makes both work: IO8 on the built-in board and 8 on the blue board are the same pin, and both are written `8` in code. AD0 and A0 are both `A0`. So wire the parts to the pin numbers your code uses, and the circuit works on either board.

What to use in the exam: whichever the lab computer has. If Pick Devices shows no blue ARDUINO UNO V3, use the built-in one through New Project → Firmware → Arduino Uno, which is the way you did your assignment.

Every component in your door-lock circuit

Your assignment circuit has 8 kinds of parts. Every one is either an **input** (the user gives information to the Arduino) or an **output** (the Arduino shows or does something).

Part (label in your circuit)	Proteus name (search with P)	Input or output	Job	How it connects
Arduino Uno (ARD1)	ARDUINO UNO	the brain	Runs the code and reads or drives every pin	Built-in: no power wires
16×2 LCD (LCD1)	LM016L	output	Shows Password, ACCESS GRANTED and so on	6 signal wires + VSS, RW to GND, VDD to +5 V, VEE to the pot
Keypad 4×3	KEYPAD-PHONE	input	User types the password	4 row wires + 3 column wires
Servo motor	MOTOR-PWMSERVO	output	Turns 0° (locked) or 90° (unlocked)	+5 V, GND, 1 signal wire
Buzzer (BUZ1)	BUZZER	output	Sounds on a wrong password	1 pin to Arduino, 1 pin to GND
Red LED (D2)	LED-RED	output	ON = door locked	Arduino pin → 220 Ω → LED → GND
Green LED (D3)	LED-GREEN	output	ON = door unlocked	Arduino pin → 220 Ω → LED → GND
Resistors (R3, R4)	RES (220 Ω)	protection	Limit LED current	In series with each LED
Potentiometer (RV1)	POT-HG (10k)	adjust	LCD contrast	Ends to +5 V and GND, middle pin to LCD VEE

The small "LED & Reset" block inside the Arduino's dashed box is part of the Arduino model. Leave it alone.

How the whole system flows: keypad → Arduino compares the typed code with "1234" → if it matches: LCD shows ACCESS GRANTED, green LED on, servo to 90°, wait 5 s, servo back to 0° → if it doesn't: LCD shows ACCESS DENIED, buzzer beeps, attempts + 1 → after

3 wrong attempts: entry blocked for 10 s.

Connection rules for any circuit

You don't need to memorise circuits. With these 9 rules you can wire any of the lab programs from its code.

1. **The code decides the wiring.** Every pin number in the code (`pinMode(10, ...)`, `lcd(10, 11, ...)`, `door.attach(A2)`) is a wire to that pin. If the code says 10, the wire goes to IO10 (or 10 on the blue board).
2. **Every part needs a complete loop:** power in, ground out. A part with only one wire does nothing. Your buzzer last night had this problem.
3. **LED:** Arduino pin → 220 Ω resistor → LED anode (flat side of the triangle) → LED cathode (bar side) → GND. Without the resistor, too much current flows.
4. **LCD (LM016L), 4-bit mode:** RS, E, D4, D5, D6, D7 go to the 6 pins named in `LiquidCrystal lcd(RS, E, D4, D5, D6, D7)`. VSS and RW go to GND, VDD to +5 V, and VEE to a pot (or GND). D0 to D3 stay empty.
5. **Keypad:** each row and each column is one wire. A 4×3 keypad needs 7 pins and a 4×4 needs 8. The pins go in the code's `rowPin` and `colPin` arrays, in order (row A first).
6. **Sensors that give a voltage** (LM35, LDR divider, pot) must go to **A0 to A5**, because only those pins can read voltage with `analogRead`.
7. **Brightness or speed control** (`analogWrite`) only works on the PWM pins **3, 5, 6, 9, 10, 11** (marked ~ on the board).
8. **Servo:** 3 wires. Red to +5 V, black/brown to GND, and the signal to the pin in `door.attach(pin)`.
9. **A0 to A5 can also act as normal digital pins.** That's why your assignment drives the LCD, servo, buzzer and LEDs from A0 to A5.

Your assignment's exact connections

These come straight from the code in your Assignment 10. On the blue board, use the same numbers without the IO and AD prefixes.

Part pin	Built-in board pin	Code line that sets it
LCD RS	IO10	`LiquidCrystal lcd(10, 11, 12, 13, A0, A1);`
LCD E	IO11	same line

Part pin	Built-in board pin	Code line that sets it
LCD D4, D5	IO12, IO13	same line
LCD D6, D7	AD0, AD1	same line
LCD VSS, RW	GND	none (always GND)
LCD VDD	+5 V	none
LCD VEE	RV1 middle pin (RV1 ends to +5 V and GND)	none
Keypad rows A, B, C, D	IO2, IO3, IO4, IO5	`rowPins[ROWS] = {2, 3, 4, 5};`
Keypad columns 1, 2, 3	IO6, IO7, IO8	`colPins[COLS] = {6, 7, 8};`
Servo signal	AD2	`door.attach(A2);`
Servo + and −	+5 V, GND	none
Buzzer	AD3, other pin to GND	`tone(A3, 1000, 500);`
Red LED (locked)	AD4 → R3 220 Ω → LED → GND	`digitalWrite(A4, HIGH);` at start
Green LED (unlocked)	AD5 → R4 220 Ω → LED → GND	`digitalWrite(A5, HIGH);` when granted

How to explain it in one breath: "The keypad uses 7 digital pins (2 to 8), the LCD uses 4-bit mode on 10 to 13 plus A0 and A1, and the servo, buzzer and two LEDs use A2 to A5 as digital outputs."

The circuit you sent last night (blue board) used **different pins**: keypad rows on 0 to 3 and columns on 4 to 6, LCD on A4, A5, A3, A2, A1, A0, servo on 9 and buzzer on 8. Both are correct, as long as each circuit runs its own matching code.

Change tasks the examiner can give you

The usual practical question is "run it, then change X". Each change is one or two lines in your assignment code.

They ask	Change this	To this
Password 4321	`String password = "1234";`	`String password = "4321";`

They ask	Change this	To this
6-digit password	`"1234"` and `entered.length() < 4`	`"123456"` and `entered.length() < 6`
5 attempts before lockout	`if (attempts >= 3)`	`if (attempts >= 5)`
Lockout 30 s	`>= 10000` and `"Wait 10 seconds"`	`>= 30000` and `"Wait 30 seconds"`
Door open 10 s	`delay(5000);` after `door.write(90);`	`delay(10000);`
Door opens fully	`door.write(90);`	`door.write(180);`
Longer, higher beep	`tone(A3, 1000, 500);`	`tone(A3, 2000, 1000);` (2000 Hz for 1000 ms)
Different message	`lcd.print("ACCESS GRANTED");`	`lcd.print("WELCOME");` (max 16 characters a line)
Show attempts left	`lcd.print(attempts);`	`lcd.print(3 - attempts);` and change the label to `"Left: "`
Add a yellow LED that blinks on each key press	(new)	Wire IO9 → 220 Ω → LED-YELLOW → GND. Add `pinMode(9, OUTPUT);` in `setup()`, and in `loop()` after `if (key != '\0') {` add `digitalWrite(9, HIGH); delay(100); digitalWrite(9, LOW);`
Use a 4×4 keypad (KEYPAD-SMALLCALC)	`COLS = 3`, `colPins {6, 7, 8}`, 4×3 `keys` table	`COLS = 4`, `colPins {6, 7, 8, 9}` (wire column 4 to IO9), and a 4×4 table: `{'7','8','9','/'}, {'4','5','6','*'}, {'1','2','3','-'}, {'C','0','=','+'}`. Then use `'C'` instead of `'*'` to clear and `'='` instead of `'#'` to enter

Circuit-only questions follow the rules above. For example, to move the buzzer to pin 9, rewire the buzzer to IO9 and change `A3` to `9` in both the `pinMode` and `tone` lines. A pin number always changes in **two** places: the wire and the code.

Viva questions with short answers

The answers are short enough to say aloud.

The board

Question	Answer
Which microcontroller is on the Arduino Uno?	ATmega328P, 8-bit, 16 MHz
Its memory?	32 KB flash (program), 2 KB SRAM (variables), 1 KB EEPROM
How many I/O pins?	14 digital (0 to 13, six of them PWM: 3, 5, 6, 9, 10, 11) and 6 analog inputs (A0 to A5)
ADC resolution?	10-bit, so `analogRead` gives 0 to 1023 for 0 to 5 V
What do `setup()` and `loop()` do?	`setup()` runs once at power-on (pin modes, LCD start). `loop()` repeats forever.
How does Proteus run the code?	The code is compiled to a .hex file (machine code), and Proteus simulates the chip running it
The question says 8051 (AT89C52), so why Arduino?	Same idea, easier tools. On an AT89C52 (8 KB flash, 256 bytes RAM, ports P0 to P3) you'd add a crystal (11.0592 or 12 MHz) with two 33 pF capacitors, a reset circuit, and pull-up resistors on port P0, and write the code in Keil.

The parts

Question	Answer
Why a 220 Ω resistor with an LED?	To limit current: (5 V − about 2 V LED drop) / 220 Ω ≈ 14 mA, under the LED's 20 mA limit
Why is LCD RW on GND?	RW = 0 means write. We only send data to the LCD and never read from it.
What do RS and E do?	RS picks command (0) or character data (1). A pulse on E makes the LCD take the data on D4 to D7.

Question	Answer
Why only D4 to D7 (4-bit mode)?	It saves pins: 6 wires instead of 10. Each byte is sent as two halves.
What is the pot on VEE for?	It sets the LCD contrast
How does the keypad work?	Rows are outputs and columns are inputs. The code pulls one row LOW at a time. A pressed key joins its row to its column, so that column reads LOW, and the row and column together give the key.
Why `INPUT_PULLUP` on the columns?	It holds each column HIGH when no key is pressed, so the pin doesn't float and give random readings
What is debouncing?	A key bounces for a few ms. The code waits 30 ms and checks again so one press counts once.
How is a servo controlled?	By a PWM pulse every 20 ms. The pulse width (about 1 to 2 ms) sets the angle. `door.write(90)` asks for 90°.
What does `tone(A3, 1000, 500)` do?	It sends a 1000 Hz square wave to the buzzer for 500 ms
Why `millis()` for the lockout and not `delay()`?	`delay()` freezes everything. `millis()` lets `loop()` keep running while it checks whether 10 s have passed.
What if the LCD is blank?	Check that RS and E reach their pins, that VSS and RW are on GND and VDD on +5 V, and that the `lcd(...)` pin order is RS, E, D4, D5, D6, D7

Other lab programs

Question	Answer
LM35 output?	10 mV per °C. T = `analogRead` × 500 / 1023.
How does an LDR street light work?	The LDR and a 10k resistor form a voltage divider. Darkness raises the LDR's resistance, so the voltage at A0 changes, and past a threshold the code turns the LED on.

Question	Answer
Common anode vs common cathode 7-segment?	Cathode: common to GND, and a segment lights on HIGH. Anode: common to +5 V, and a segment lights on LOW.
What is PWM?	Switching a pin on and off fast. The ON fraction (duty cycle) sets the average power: `analogWrite(pin, 0 to 255)`.